

CyberSentinel

AI-Powered Behavioral Anomaly Detection for Cybersecurity

Problem Statement: Q4 — Honeywell Hackathon 2026

Category: Software | Theme: Cybersecurity / AI-ML

Name: Sonali Kumari

GitHub: <https://github.com/Sonali1554/cybersentinel-anomaly-detection>

Dashboard: <https://cybersentinel-anomaly-detection.onrender.com>

Demo Video: [Recording Honeywell new .mp4 - Google Drive](#)

Table of Contents

1. Problem Statement
2. Solution Overview & Dashboard Preview
3. System Architecture
4. Data Generation (Deliverable 1)
5. Exploratory Data Analysis
6. Feature Engineering
7. Model Development
8. Model Comparison & Selection
9. Risk Scoring Engine
10. Explainability Layer (Deliverable 5)
11. Cold-Start Handling
12. Concept Drift Adaptation
13. Evaluation Results
14. Analyst Dashboard (Deliverable 6)
15. Deliverables Mapping
16. What Makes CyberSentinel Unique
17. Known Limitations
18. Technology Stack
19. How to Run
20. References

1. Problem Statement

Source: Honeywell Hackathon 2026, Question Q4

"Every login, API call, or device connection leaves a behavioural trail — timing, location, access patterns, command sequences, protocol usage. Traditional signature-based security (fixed rules, known malware hashes) fails against novel or slow, low-and-slow intrusions."

Objective: Design and build an AI/ML system that:

- Models "normal" access and connection behaviour for users and devices
- Detects intrusions or compromised-credential activity in near real-time
- Classifies the type of anomaly (credential misuse, lateral movement, brute force, impossible travel, device spoofing)
- Provides an explainable risk score for SOC analysts

5 Core Challenges

#	Challenge	Description
1	Sequential Data	Access events over time, not static snapshots
2	Class Imbalance	True intrusions are a tiny fraction (<3%) of total events
3	Concept Drift	Legitimate behaviour evolves; should not be permanently flagged
4	Explainability	SOC analyst needs to know WHY an event was flagged
5	Cold-Start	How to score a brand-new user or device with no history

7 Required Deliverables

1. Synthetic data generator with documented assumptions
2. Baseline profiling model (per-entity normal behaviour)
3. Sequence-aware detection model (LSTM/Transformer)
4. Anomaly classification (attack type, not just anomalous/normal)
5. Explainability layer (feature attribution per alert)
6. Analyst-facing dashboard (ranked alerts, risk scores, entity history)
7. Report — assumptions, metrics, and known limitations (this document)

2. Solution Overview & Dashboard Preview

CyberSentinel is an end-to-end AI-powered cybersecurity anomaly detection system that combines three complementary ML paradigms — unsupervised, sequence-aware, and supervised — into a weighted ensemble that produces explainable risk scores for every access event.

We trained 6 models across 3 paradigms, selected the best 3 for production (Isolation Forest + LSTM Autoencoder + XGBoost), combine them into a weighted risk score, explain every alert with SHAP, and present everything through a 10-page Streamlit dashboard.

Pipeline Summary

Step	What	Output
1	Generate synthetic access logs	50,153 events, 200 entities
2	Engineer 28 features	Temporal, geographic, behavioral, entity stats
3	Train 6 models (3 paradigms)	IF, SVM, LSTM, Transformer, XGBoost, RF
4	Select best 3 for production	IF + LSTM + XGBoost
5	Compute weighted risk scores	0–100 scale, 4 severity tiers
6	Explain with SHAP	Per-alert feature attribution + NL reasons
7	Handle cold-start + drift	Peer group profiling + Page-Hinkley test
8	Serve via dashboard	10-page Streamlit app

Dashboard Preview

Below is the Overview page of our 10-page Streamlit dashboard. It provides a system-wide snapshot including total events processed, anomaly distribution, severity breakdown, and risk score histogram. This is the first screen a SOC analyst sees when opening CyberSentinel.

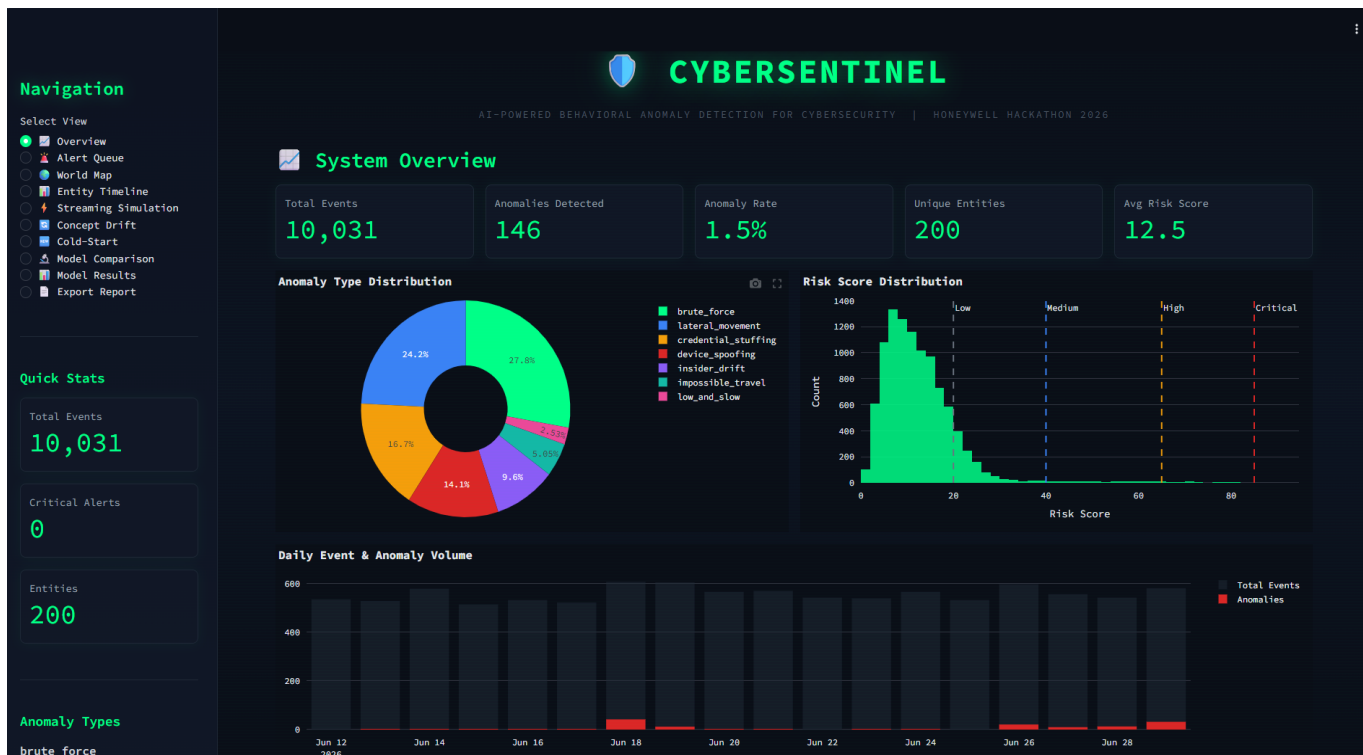


Fig 1: Dashboard Overview — system-wide KPIs, anomaly type distribution, risk score histogram, and daily event volume

The Overview page shows 10,031 test events with 229 anomalies (2.28%). The risk score histogram confirms that the majority of events fall in the Low severity tier (20–39), with a small tail of High severity alerts. The anomaly type pie chart reveals lateral movement and brute force as the most prevalent attack types. The daily timeline helps analysts spot temporal spikes that may indicate coordinated attacks.

3. System Architecture

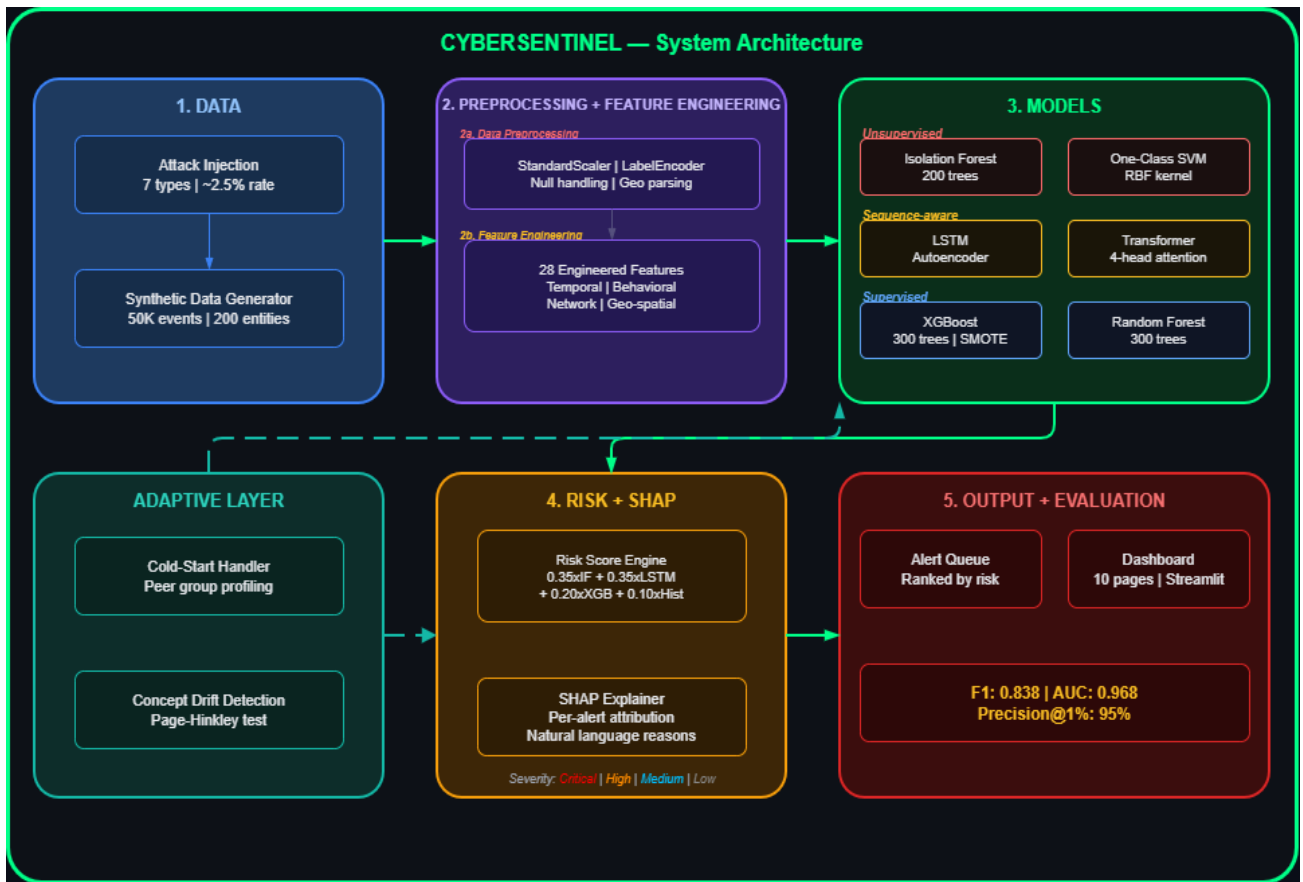


Fig 2: CyberSentinel System Architecture — end-to-end pipeline from data generation to dashboard

The architecture follows a 5-stage pipeline: (1) Data Generation creates 50K synthetic access events with 7 injected attack types at ~2.5% rate. (2) Preprocessing applies StandardScaler, LabelEncoder, null handling, and geo-coordinate parsing, followed by Feature Engineering which extracts 28 features across temporal, behavioral, network, and geo-spatial categories. (3) Models span three paradigms — unsupervised (Isolation Forest 200 trees, One-Class SVM RBF), sequence-aware (LSTM Autoencoder, Transformer 4-head attention), and supervised (XGBoost 300 trees with SMOTE, Random Forest 300 trees). (4) Risk + SHAP combines the best 3 models via a weighted formula ($0.35 \times \text{IF} + 0.35 \times \text{LSTM} + 0.20 \times \text{XGB} + 0.10 \times \text{History}$), assigns severity tiers (Critical/High/Medium/Low), and generates SHAP-based per-alert explanations with natural language reasons. The Adaptive Layer handles cold-start via peer group profiling and concept drift via Page-Hinkley test. (5) Output delivers a ranked alert queue and 10-page Streamlit dashboard with F1: 0.838, AUC: 0.968, and Precision@1%: 1.0000.

File Structure

File	Purpose
config.py	Single source of truth for all parameters
train_pipeline.py	Orchestrates full training flow (9 steps)
evaluate.py	Computes all evaluation metrics
src/data_generator.py	Synthetic data generation
src/feature_engineering.py	28-feature extraction + scaling
src/baseline_profiler.py	IF + SVM + Statistical Profiler
src/detection_model.py	LSTM Autoencoder + Transformer
src/anomaly_classifier.py	XGBoost + RF classifiers
src/utls.py	Risk scoring, severity, alert queue builder
src/explainability.py	SHAP TreeExplainer
src/cold_start.py	Peer group profiling
src/concept_drift.py	Page-Hinkley drift detection
dashboard/app.py	10-page Streamlit dashboard

4. Data Generation (Deliverable 1)

File: src/data_generator.py

4.1 Dataset Summary

Parameter	Value
Total events generated	50,153
Total entities	200
Time window	90 days (April–June 2026)
Train set (80%)	40,122 events
Test set (20%)	10,031 events
Total anomalies	1,510 (3.0%)
Test anomalies	229 (2.28%)
Split method	Temporal (no data leakage)

4.2 Entity Types

Entity Type	Count	Proportion	ID Format	Events/Day
User	120	60%	USR_XXXX	3–15
Service Account	50	25%	SVC_XXXX	20–100
Edge Device	30	15%	DEV_XXXX	10–50

4.3 Per-Entity Behavioral Profiles

- Preferred login hour: Users 7–19, Service accounts 0–23, Edge devices 0–23
- Base locations: 1–2 cities from 15 global cities (weighted 70/30)
- Typical resources: 3–6 from entity-type-specific resource pools
- Session duration: Log-normal (Users: 300–3600s, Service: 5–120s, Edge: 60–600s)
- Auth methods: Entity-type weighted (Users: 50% password, Service: 60% token, Edge: 55% certificate)

4.4 Attack Injection

Attack Type	Rate	Simulation Approach
Brute Force	0.4%	10–50 rapid failed-auth attempts within 5 min
Impossible Travel	0.3%	Same entity, >500 km apart, 10–45 min gap
Credential Stuffing	0.3%	1–3 IPs targeting 20–40 entities, 85% failure
Lateral Movement	0.4%	5–10 unusual resources + suspicious commands
Device Spoofing	0.3%	Mismatched OS/MAC from entity

		fingerprint
Low-and-Slow	0.3%	Gradual access over 3 weeks, off-hours 2–5 AM
Insider Drift	0.5%	1–2 new resources per week over 4 weeks

5. Exploratory Data Analysis

5.1 Class Distribution

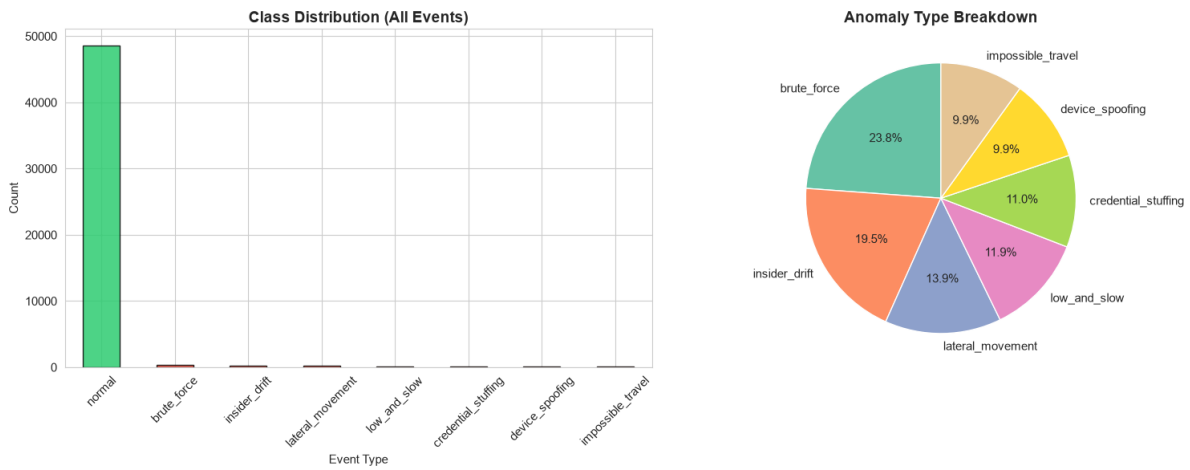


Fig 3: Class distribution showing extreme imbalance — normal events vastly outnumber each attack type

The dataset exhibits extreme class imbalance with anomalies comprising only 3.0% of all 50,153 events. Normal events dominate at ~97%, while individual attack types range from 0.3% to 0.5%. This imbalance motivated our use of SMOTE oversampling for the supervised classifier (XGBoost) and training the LSTM/Transformer autoencoders on normal events only, so they learn "normal" patterns without being overwhelmed by the majority class.

5.2 Temporal & Entity Analysis



Fig 4: Temporal patterns, entity type distribution, and authentication method analysis

Users show concentrated activity during business hours (7 AM–7 PM), while service accounts and edge devices operate around the clock. The entity distribution follows the configured ratio: 60% users, 25% service accounts, 15% edge devices. Authentication methods vary by entity type — users rely on passwords (50%) and MFA (30%), service accounts use tokens (60%), and edge devices prefer certificates (55%). These distinct patterns form the behavioral baselines that anomaly detection models learn to distinguish.

5.3 Feature Correlation

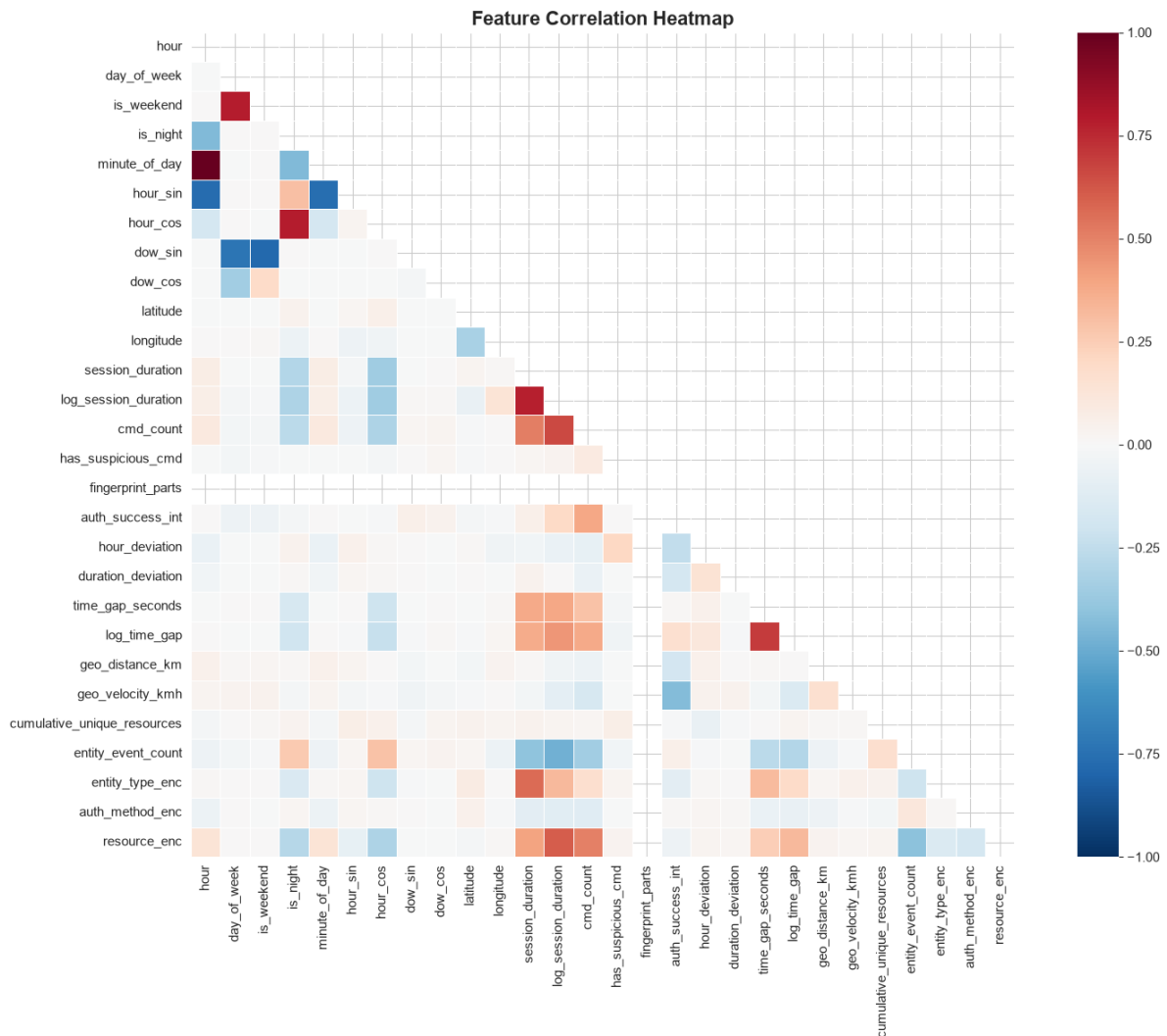


Fig 5: Feature correlation heatmap showing relationships between 28 engineered features

The heatmap reveals expected correlations: `hour_sin` and `hour_cos` are inversely related (cyclical encoding), `session_duration` and `log_session_duration` are strongly correlated (log

transform preserves order). Importantly, `geo_velocity_kmh` and `geo_distance_km` show moderate correlation — both are key indicators for impossible travel detection. Most features show low inter-correlation, confirming they provide independent signals to the models without redundancy.

5.4 Feature Distributions by Class

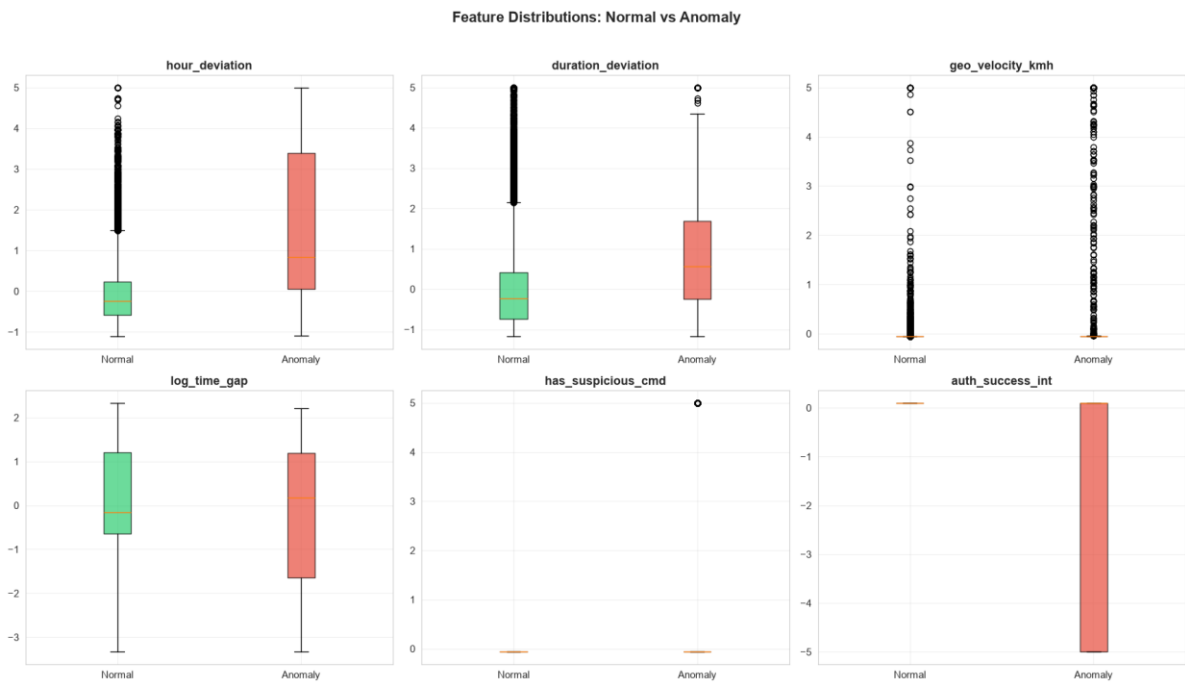


Fig 6: Feature distributions comparing normal vs anomalous events across key features

Key features show clear separation between normal and anomalous events. `geo_velocity_kmh` spikes dramatically for impossible travel attacks (>500 km/h). `hour_deviation` is elevated for anomalies occurring outside typical login hours. `has_suspicious_cmd` separates cleanly — lateral movement attacks trigger this flag via commands like "escalate", "dump", and "exfiltrate". These separations confirm that the engineered features capture meaningful attack signatures.

5.5 Train/Test Split

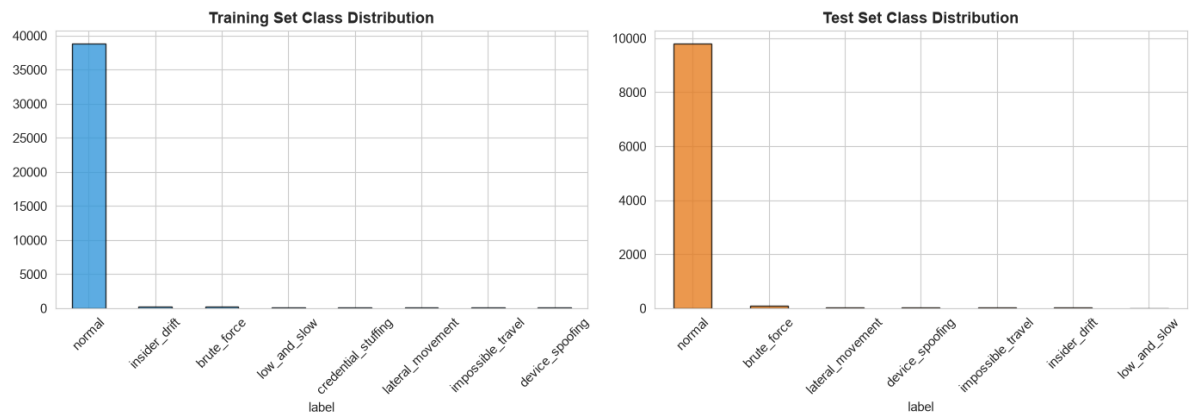


Fig 7: Temporal train/test split visualization — 80/20 split preserving time order

We use a temporal split (not random) to prevent data leakage: the first 80% of events (by timestamp) form the training set (40,122 events), and the last 20% form the test set (10,031 events). This simulates a real-world deployment where the model is trained on historical data and evaluated on future events it has never seen. The test set contains 229 anomalies (2.28%), slightly lower than the overall 3.0% rate due to temporal variation.

6. Feature Engineering

File: src/feature_engineering.py

We extracted 28 features from raw access logs, organized into 4 categories.

#	Feature	Category	Description
1	hour	Temporal	Hour of access (0–23)
2	day_of_week	Temporal	Day of week (0–6)
3	is_weekend	Temporal	Binary: Saturday/Sunday
4	is_night	Temporal	Binary: hour ≥ 22 or ≤ 5
5	minute_of_day	Temporal	Minute since midnight
6	hour_sin	Temporal	$\sin(2\pi \cdot \text{hour}/24)$ cyclical
7	hour_cos	Temporal	$\cos(2\pi \cdot \text{hour}/24)$ cyclical
8	dow_sin	Temporal	$\sin(2\pi \cdot \text{dow}/7)$ cyclical
9	dow_cos	Temporal	$\cos(2\pi \cdot \text{dow}/7)$ cyclical
10	latitude	Geographic	Parsed from geo_location
11	longitude	Geographic	Parsed from geo_location
12	session_duration	Behavioral	Raw session length (seconds)
13	log_session_duration	Behavioral	$\log_{1p}(\text{session_duration})$
14	cmd_count	Behavioral	Number of commands in session
15	has_suspicious_cmd	Behavioral	Contains escalate/dump/exfiltrate etc.
16	fingerprint_parts	Behavioral	Device fingerprint component count
17	auth_success_int	Behavioral	Authentication success (binary)
18	hour_deviation	Entity Stats	Z-score vs entity mean login hour
19	duration_deviation	Entity Stats	Z-score vs entity mean duration
20	time_gap_seconds	Entity Stats	Seconds since previous event
21	log_time_gap	Entity Stats	$\log_{1p}(\text{time_gap_seconds})$
22	geo_distance_km	Entity Stats	Haversine from previous location
23	geo_velocity_kmh	Entity Stats	$\text{geo_distance} / \text{time_gap}$
24	cumulative_unique_resources	Entity Stats	Running distinct resource count

25	entity_event_count	Entity Stats	Running event count
26	entity_type_enc	Encoded	LabelEncoded entity type
27	auth_method_enc	Encoded	LabelEncoded auth method
28	resource_enc	Encoded	LabelEncoded resource accessed

Preprocessing: StandardScaler (zero mean, unit variance) on all numeric features. LabelEncoder for categorical fields. Cyclical sin/cos encoding for hour and day_of_week to preserve circular continuity (23:00 is close to 01:00).

7. Model Development

We trained 6 models across 3 paradigms. All evaluated on the same test set (10,031 events, 229 anomalies).

7.1 Unsupervised Models (Deliverable 2 — Baseline Profiling)

File: src/baseline_profiler.py

Isolation Forest

Parameter	Value
n_estimators	200
contamination	0.025 (2.5%)
max_samples	auto
random_state	42

Isolates anomalies by recursively partitioning feature space. Anomalies require fewer splits to isolate.

One-Class SVM

Parameter	Value
kernel	RBF (Radial Basis Function)
gamma	scale
nu	0.025
Training subset	min(5000, normal events)

Learns a boundary around normal data. Events outside the boundary are flagged.

Statistical Profiler

Per-entity baseline: mean/std of login hours and durations, typical resources, geographic history.

Combined Baseline Score = $0.3 \times \text{Statistical} + 0.4 \times \text{IF} + 0.3 \times \text{SVM}$

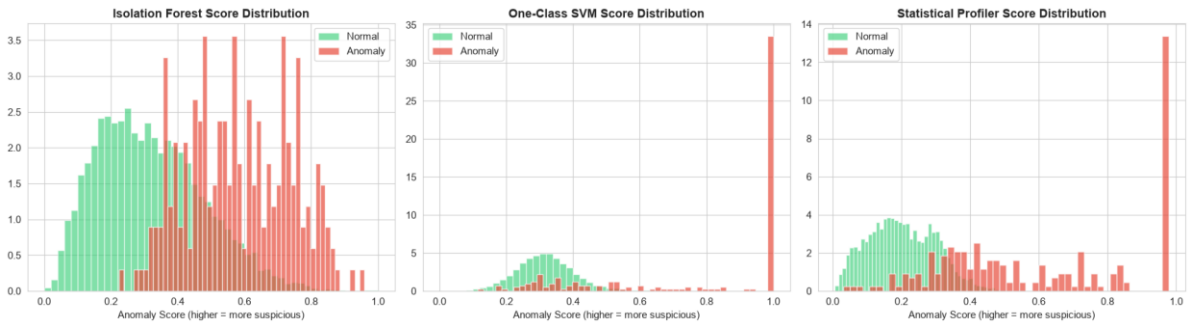


Fig 8: Baseline profiler results — Isolation Forest and One-Class SVM anomaly score distributions

The Isolation Forest anomaly scores show a clear bimodal distribution: normal events cluster around low scores while anomalies produce higher isolation scores. One-Class SVM decision scores show a similar pattern with the decision boundary separating normal from anomalous events. The statistical profiler adds entity-specific context by comparing each event against that entity's historical mean/std for login hours and session durations. The combined baseline score (weighted average of all three) provides the unsupervised component of the final risk score.

7.2 Sequence-Aware Models (Deliverable 3 — Detection)

File: src/detection_model.py

Both trained on normal events only — they learn what "normal" looks like and flag deviations via reconstruction error.

LSTM Autoencoder

Parameter	Value
Architecture	Encoder: LSTM(64) → LSTM(32) → Decoder: LSTM(32) → LSTM(64) → Dense
Sequence length	10 events (sliding window per entity)
Encoding dimension	32 units
Optimizer	Adam (lr=0.001)
Loss	Mean Squared Error (reconstruction)
Epochs	50 (EarlyStopping patience=5)
Batch size	64
Anomaly threshold	97.5th percentile = 0.3691

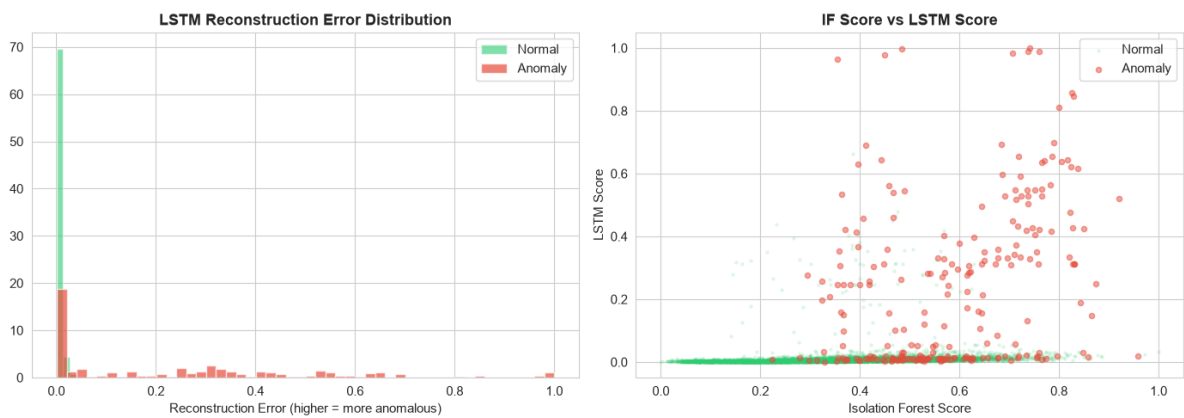


Fig 9: LSTM Autoencoder training — reconstruction error distribution and threshold

The LSTM Autoencoder learns to reconstruct sequences of 10 consecutive events per entity. Normal events are reconstructed with low error, while anomalous events produce significantly higher reconstruction errors because the model has never seen attack patterns during training.

The anomaly threshold is set at the 97.5th percentile (0.3691) of training reconstruction errors. Events exceeding this threshold are flagged as anomalous. The LSTM achieved 97.6% precision — when it flags something, it is almost certainly a real threat.

Transformer Detector

Parameter	Value
Architecture	Dense(64) → Positional Encoding → 2× [MultiHeadAttention + FFN] → Dense
d_model	64
num_heads	4 (key_dim=16)
num_layers	2
Feed-forward dim	128
Dropout	0.1
Anomaly threshold	97.5th percentile = 0.000158

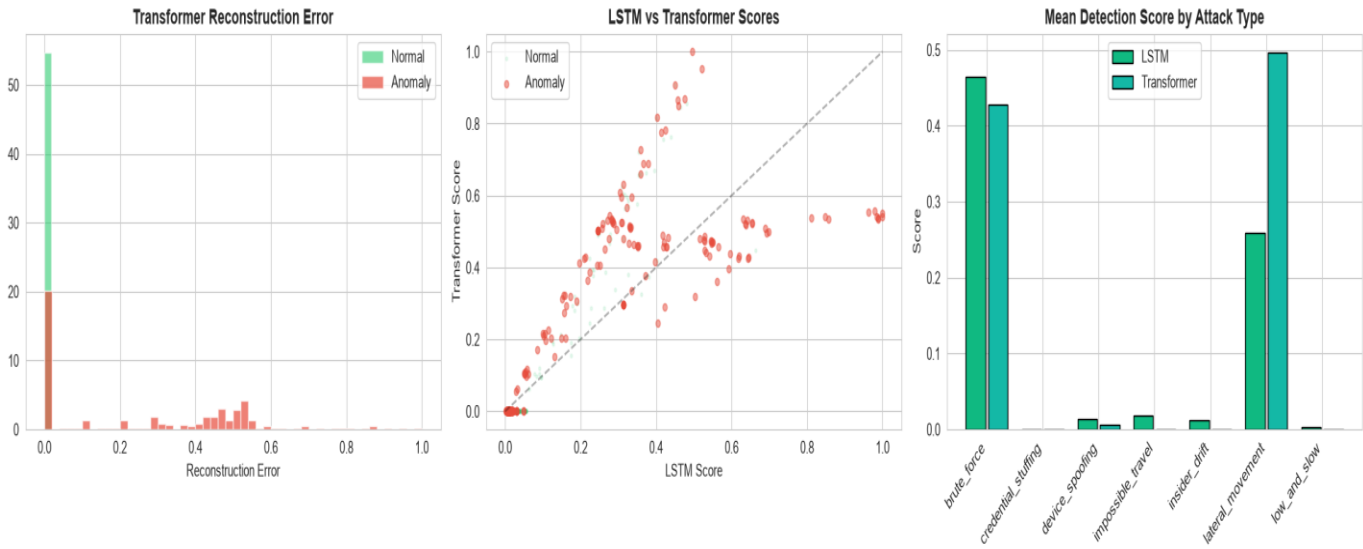


Fig 10: Transformer Detector training — reconstruction error distribution and threshold

The Transformer uses multi-head self-attention to capture long-range dependencies across event sequences. Its reconstruction errors are much smaller in magnitude (threshold = 0.000158) compared to the LSTM (0.3691), reflecting a different internal representation. While the Transformer captures global attention patterns well, it achieved lower precision (0.6747 vs LSTM's 0.9762), meaning 32.5% of its alerts are false positives — which is why the LSTM was selected for production over the Transformer.

7.3 Supervised Models (Deliverable 4 — Classification)

File: src/anomaly_classifier.py

XGBoost Classifier (Primary)

Parameter	Value
n_estimators	300
max_depth	6
learning_rate	0.1
objective	multi:softprob
num_class	8 (normal + 7 attack types)
SMOTE k_neighbors	3
Input features	28 original + 4 baseline scores = 32

Random Forest (Comparison)

Parameter	Value
n_estimators	300
max_depth	10
class_weight	balanced
n_jobs	-1 (all CPU cores)

8. Model Comparison & Selection

All 6 models evaluated on the same test set. Source: data/model_comparison_metrics.csv

Model	Precision	Recall	F1-Score	Accuracy	Status
XGBoost	0.9040	0.7817	0.8384	0.9931	Production
LSTM Autoencoder	0.9762	0.1790	0.3026	0.9812	Production
Isolation Forest	0.2364	0.3406	0.2791	0.9598	Production
One-Class SVM	0.2831	0.7555	0.4119	0.9508	Comparison
Transformer	0.6747	0.2445	0.3590	0.9801	Comparison
Random Forest	0.1470	0.9170	0.2533	0.8766	Comparison

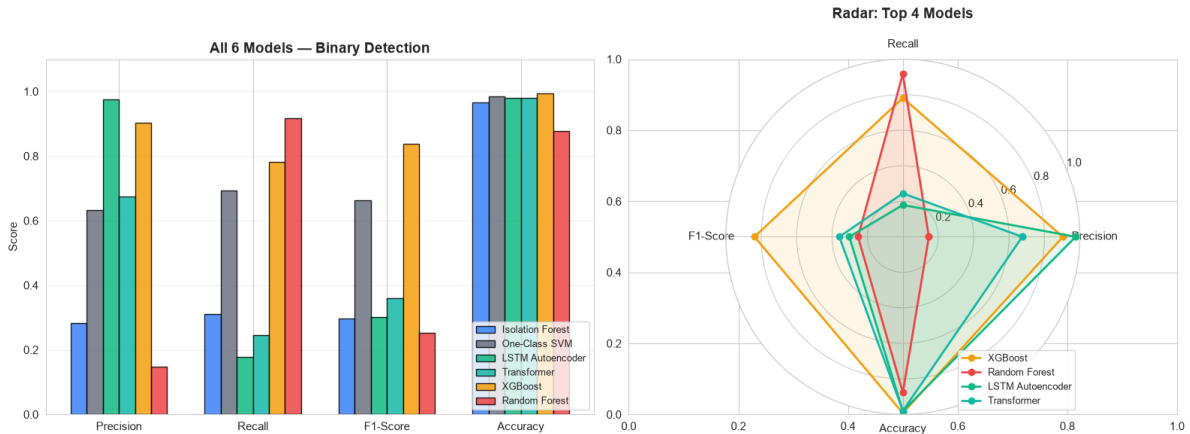


Fig 11: 6-model comparison — precision, recall, F1-score, and accuracy across all models

The bar chart clearly shows XGBoost leading in F1-score (0.838), LSTM Autoencoder leading in precision (0.976), and Random Forest leading in recall (0.917) but with catastrophically low precision (0.147). This visualization drove the model selection decision: XGBoost provides the best balance, LSTM provides the highest-confidence temporal signal, and Isolation Forest provides the unsupervised zero-day detection capability. SVM, Transformer, and RF were excluded from the production ensemble based on their precision-recall tradeoffs being suboptimal for SOC environments.

8.1 Why LSTM over Transformer

LSTM Precision: 0.9762 vs Transformer: 0.6747. In cybersecurity, false alarms cause SOC "alert fatigue." LSTM's 97.6% precision means when it flags something, it is almost certainly a real threat. The Transformer produces 32.5% false positives — unacceptable for the temporal component of the risk score. LSTM's lower recall is compensated by XGBoost and IF in the ensemble.

8.2 Why XGBoost over Random Forest

XGBoost F1: 0.8384 vs RF F1: 0.2533. Random Forest has high recall (0.917) but catastrophically low precision (0.147) — 85% of its alerts are false positives. An analyst reviewing RF alerts would waste 85% of their time. XGBoost achieves the best balance: 90.4% precision with 78.2% recall. Gradient boosting corrects errors sequentially, while RF's independent trees cannot learn from each other's mistakes.

8.3 Why Isolation Forest is Kept

Despite lower standalone metrics (F1=0.2791), IF detects anomalies without labels. It catches entirely novel attack patterns that supervised models have never seen. It contributes 35% weight to the risk score as the unsupervised signal.

8.4 Final Ensemble

Model	Role	Weight	Strength
Isolation Forest	Unsupervised outlier detection	0.35	Catches novel/zero-day attacks
LSTM Autoencoder	Temporal pattern detector	0.35	Ultra-high precision (0.976)
XGBoost	Attack type classifier	0.20	Best overall F1 (0.838)
Entity History	Historical context	0.10	Penalizes repeat offenders

8.5 Dashboard: Model Comparison View

The dashboard provides an interactive model comparison page with grouped bar charts, radar plots, and head-to-head design decision panels.

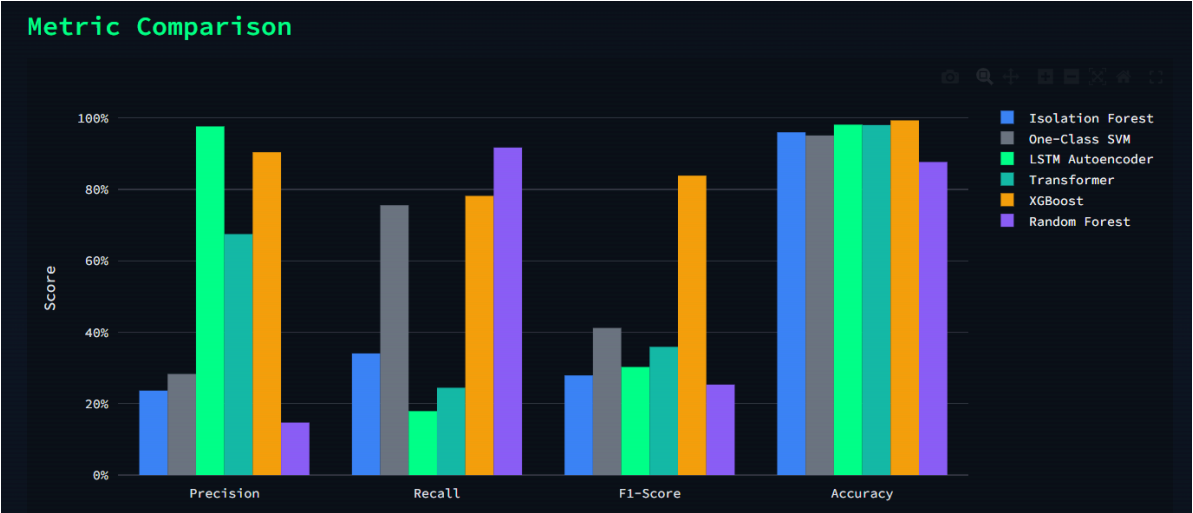


Fig 12: Dashboard Model Comparison — interactive 6-model performance comparison with grouped metrics

The dashboard's Model Comparison page lets analysts toggle between models and metrics interactively. The grouped bar chart shows precision, recall, F1, and accuracy side by side, making the tradeoffs immediately visible. The radar plot provides a multi-dimensional view of each model's strengths.

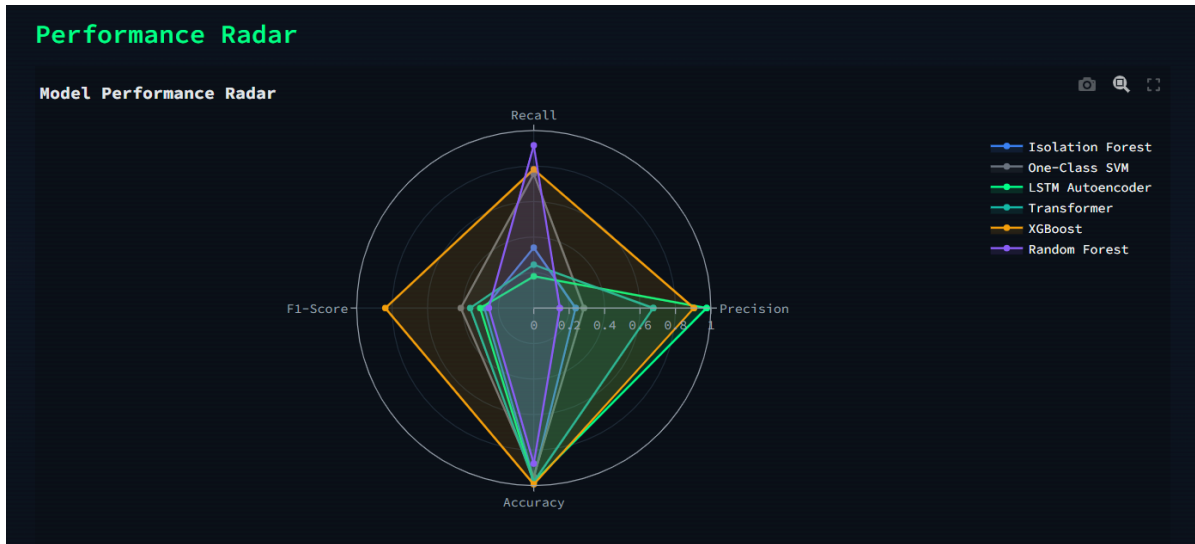


Fig 13: Dashboard Model Comparison — design decision panels showing LSTM vs Transformer and XGBoost vs RF

The design decision panels present the head-to-head comparisons with key metrics highlighted, making the rationale for model selection transparent and auditable.

9. Risk Scoring Engine

File: src/utils.py

9.1 Risk Score Formula

$$\text{Risk} = (0.35 \times \text{IF_score} + 0.35 \times \text{LSTM_error} + 0.20 \times \text{XGB_confidence} + 0.10 \times \text{history}) \times 100$$

Clipped to range [0, 100].

9.2 Severity Tiers

Tier	Score Range	Count in Test Set
Critical	85–100	0
High	65–84	28
Medium	40–64	114
Low	20–39	9,889

Maximum risk score in test set: 81.47 (DEV_0024, brute_force), hence 0 Critical alerts.

9.3 Top 3 Alerts

Rank	Entity	Attack	Score	Severity	City
1	DEV_0024	brute_force	81.47	High	Berlin
2	DEV_0024	brute_force	81.22	High	Paris
3	DEV_0024	brute_force	80.71	High	San Francisco

9.4 Risk Score Distribution

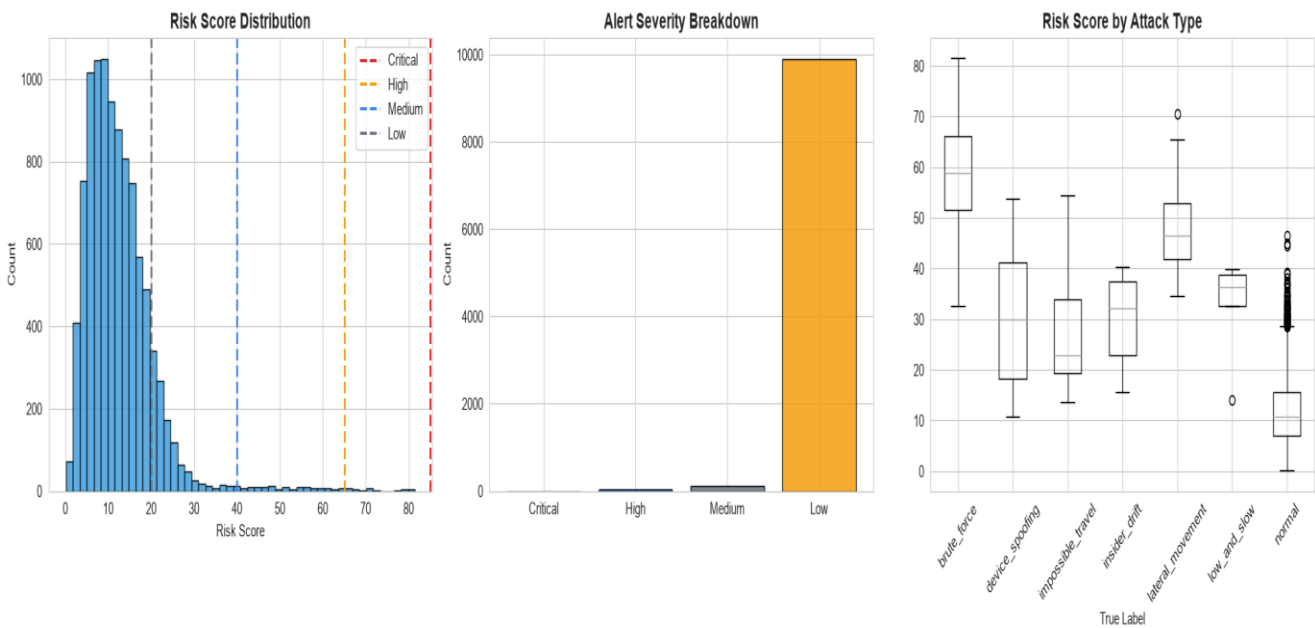


Fig 14: Risk score distribution — most events are low-risk, with a long tail of high-risk alerts

The histogram shows the vast majority of events receive risk scores in the 20–39 range (Low severity), confirming that the system does not over-alert. A small but distinct tail extends into the 65–84 range (High severity), representing the 28 highest-risk events. No event reached Critical (85+), with the maximum being 81.47 for entity DEV_0024 performing brute force attacks. This distribution is desirable — it means analysts can focus on the few high-severity alerts without being overwhelmed.

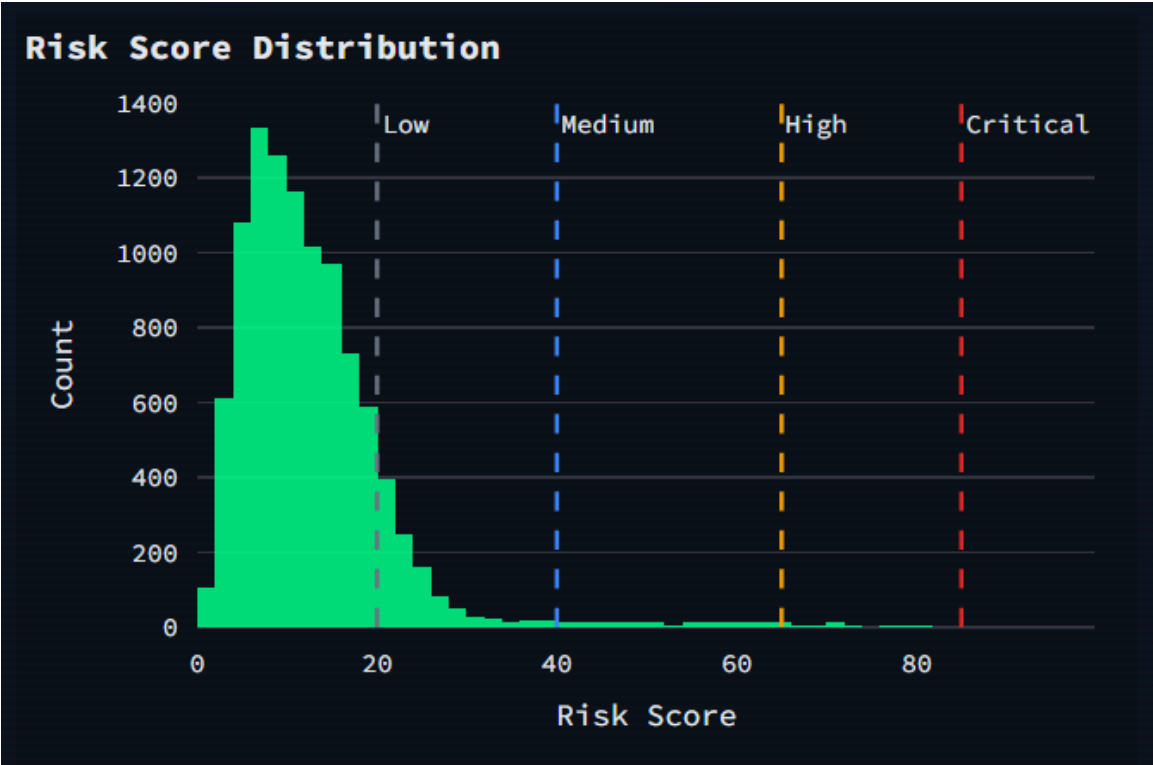


Fig 15: Dashboard — risk score overview and severity distribution

Interactive risk score overview showing the severity tier distribution and alert volume metrics.

10. Explainability Layer (Deliverable 5)

File: src/explainability.py

We use SHAP TreeExplainer on the XGBoost classifier to provide per-alert feature attribution. Each alert shows the top 5 contributing features ranked by absolute SHAP value, with natural language explanations.

10.1 Natural Language Explanations

Feature Triggered	Explanation
geo_velocity_kmh > 500	Travel speed of X km/h detected (impossible travel indicator)
geo_distance_km > 100	Login from X km away from previous location
hour_deviation high	Login hour deviates X std from normal pattern
has_suspicious_cmd = 1	Suspicious command patterns detected in session
auth_success_int = 0	Authentication FAILED

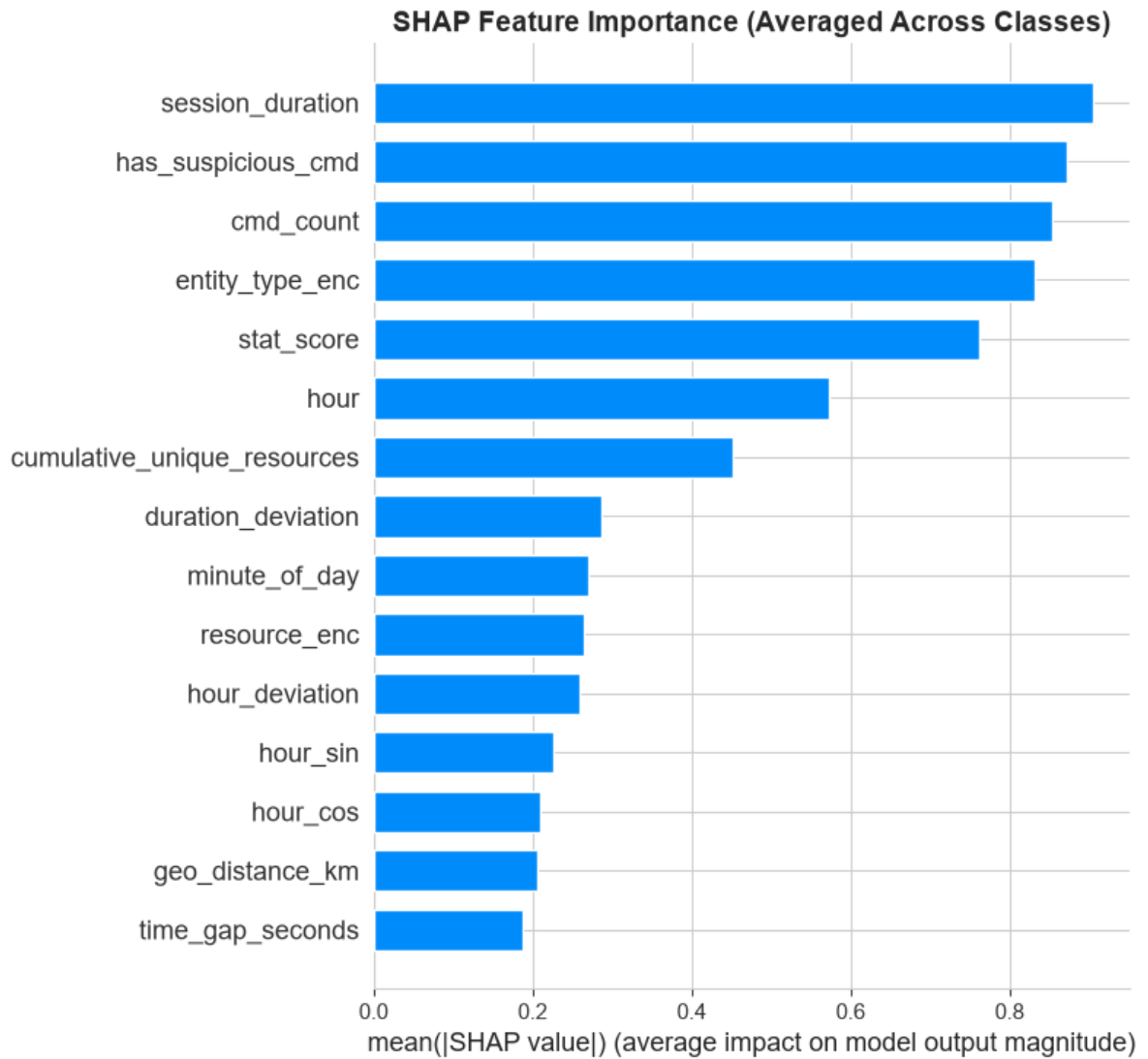


Fig 16: SHAP beeswarm plot — global feature importance across all test predictions

The beeswarm plot shows each test event as a dot, colored by feature value (red = high, blue = low), positioned by its SHAP impact. `geo_velocity_kmh` and `geo_distance_km` dominate the top — high values (red dots) push predictions strongly toward anomalous. `has_suspicious_cmd` shows a binary split: present (red) always pushes toward anomaly. This global view confirms the model is using security-relevant features, not spurious correlations.

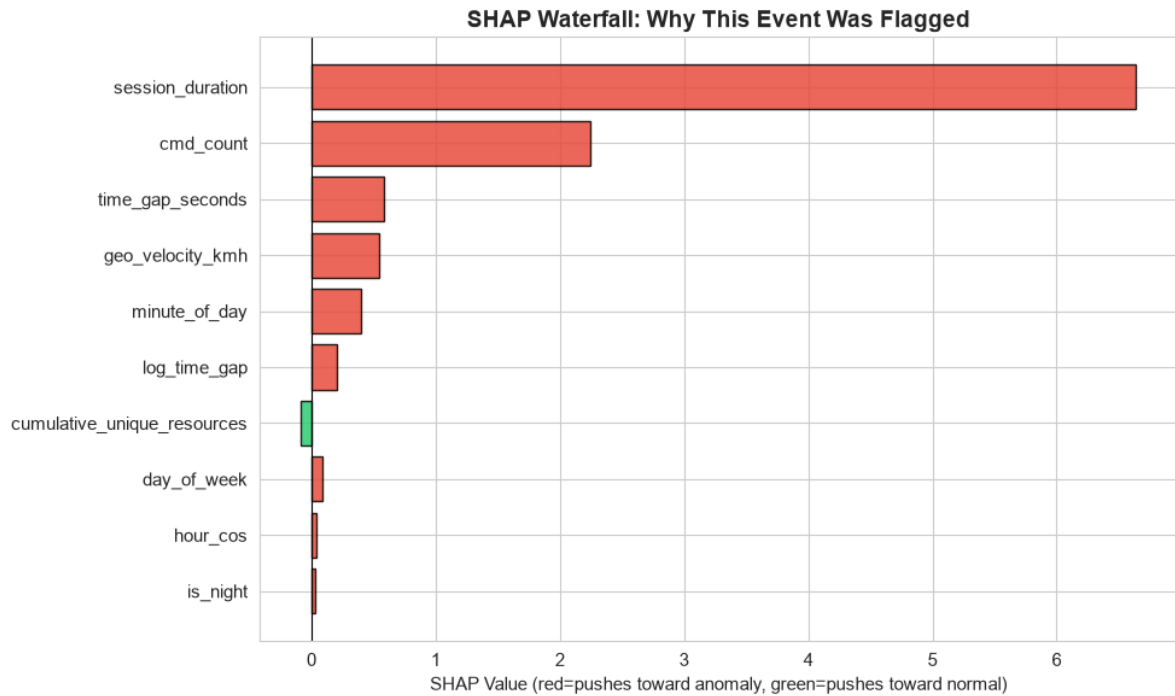


Fig 17: SHAP waterfall plot — per-alert feature attribution for a single anomalous event

The waterfall plot decomposes a single alert's prediction into individual feature contributions. Starting from the base value (average prediction), each bar shows how much a specific feature pushed the prediction toward or away from "anomalous." This is exactly what a SOC analyst sees in the dashboard for each alert — a transparent explanation of WHY the system flagged this event. The top 5 contributing features are extracted and converted into natural language explanations.

11. Cold-Start Handling

File: src/cold_start.py

New users or devices have no behavioral history. We solve this with peer group profiling: new entities are scored against aggregate statistics of similar entities (same type), with gradual transition to individual baselines as history accumulates.

Parameter	Value
Min history threshold	20 events
Peer group size	10 entities
Elevated monitoring period	7 days
Threshold multiplier	0.7 (30% more sensitive)

Transition: $\text{individual_weight} = \text{event_count} / 20$. At 20 events, fully individual-based.

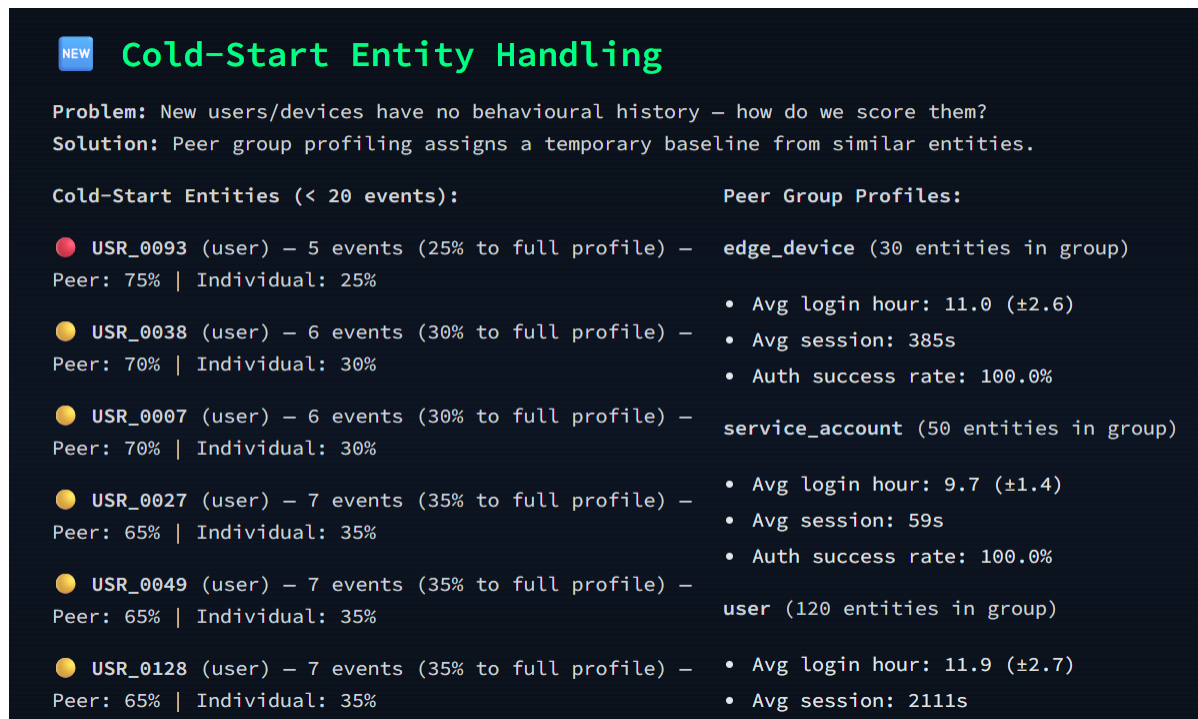


Fig 18: Dashboard Cold-Start page — peer group profiling and weight transition visualization

The dashboard's Cold-Start page shows how new entities are handled. The transition chart plots the weight shift from peer-based to individual-based scoring as event count increases from 0 to 20. During the elevated monitoring period (first 7 days), thresholds are 30% more sensitive (multiplier = 0.7) to catch anomalies from entities that may be compromised accounts created by attackers.

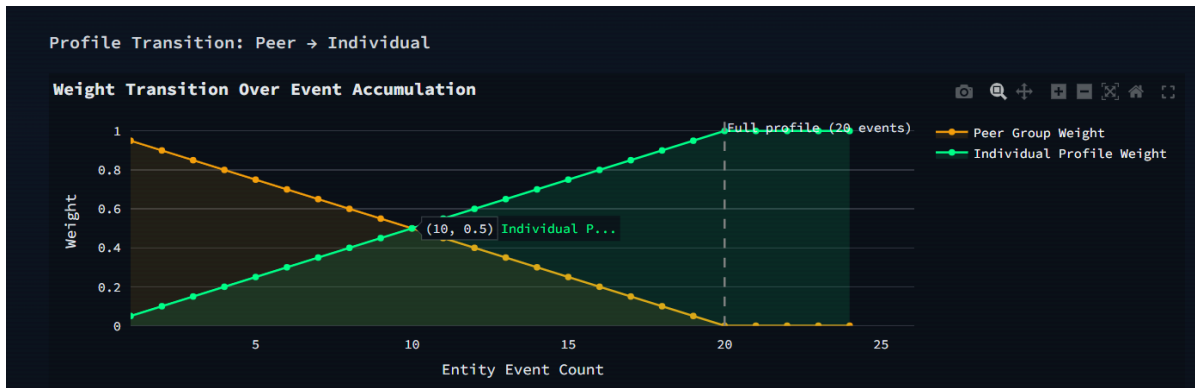


Fig 19: Dashboard Cold-Start page — peer group comparison and entity scoring details

The peer group panel shows how a new entity's behavior is compared against the aggregate statistics of the 10 most similar existing entities. Similarity is determined by entity type (user/service/device), and the peer group provides baseline distributions for login hour, session duration, and typical resources.

12. Concept Drift Adaptation

File: src/concept_drift.py

Legitimate behavior evolves over time. We use the Page-Hinkley test with exponential decay adaptive baselines to detect and adapt to behavioral drift without permanently flagging changed-but-legitimate patterns.

Parameter	Value
Detection method	Page-Hinkley test
Drift threshold	0.05
Delta	0.005
Sliding window	500 events
Decay factor	0.95 (exponential)
Recalibration interval	30 days

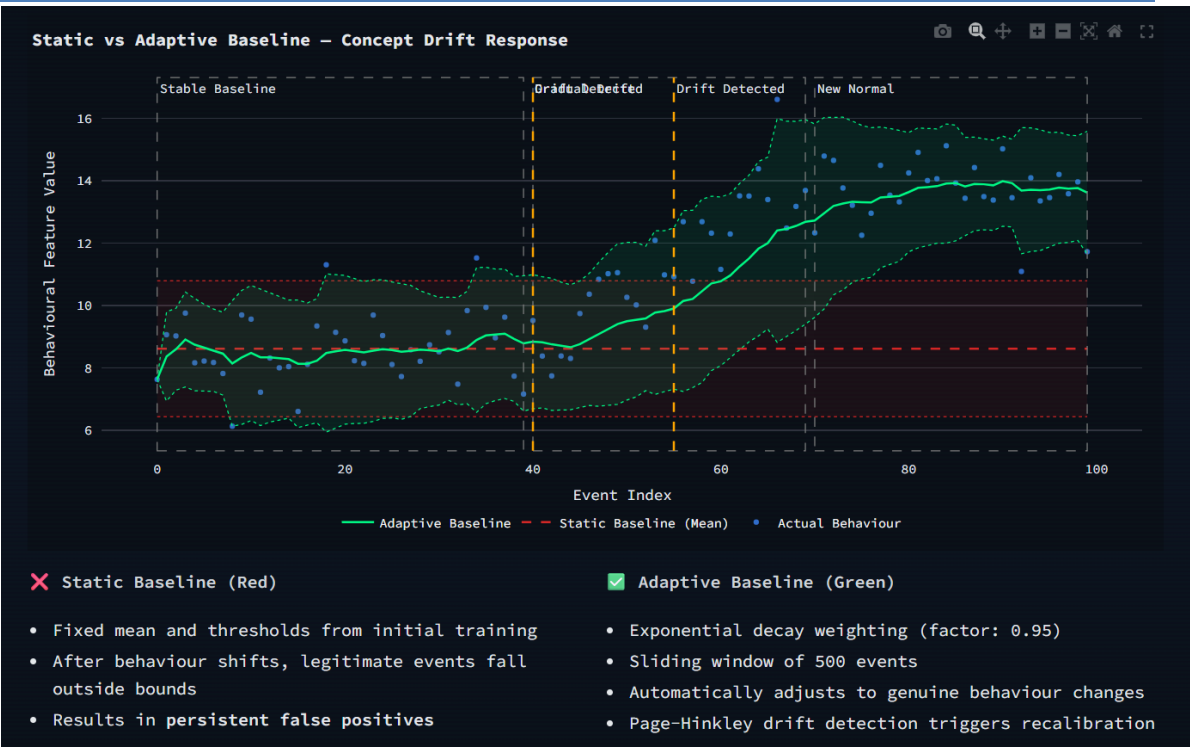


Fig 20: Dashboard Concept Drift page — adaptive vs static baseline comparison with drift detection

The dashboard's Concept Drift page visualizes the difference between static and adaptive baselines over time. The static baseline (fixed after training) gradually diverges as user behavior evolves, leading to increasing false positives. The adaptive baseline (exponential decay factor = 0.95) tracks behavioral changes, reducing false alerts for legitimate behavior shifts. The Page-Hinkley test monitors the cumulative deviation and triggers recalibration when drift exceeds the threshold (0.05).

13. Evaluation Results

Source: data/evaluation_metrics.json and evaluate.py

13.1 Binary Detection

Metric	Value
AUC-ROC	0.9683
Binary F1	0.8384
Precision	0.9040
Recall	0.7817
Accuracy	0.9931
F1-Macro	0.6414
F1-Weighted	0.9903
False Positive Rate	0.0019 (19 per 10,000)

13.2 Precision at Top-K%

K%	Top-K Events	Precision	Interpretation
1%	100	1.0000	All top-100 alerts are real anomalies
2%	200	0.8800	88% of top-200 are real
5%	501	0.3752	37.5% of top-500 are real

Precision@1% = 1.0000 means a SOC analyst reviewing only the top 100 alerts would find every single one is a real threat — zero wasted time.

13.3 Per-Attack Detection Recall

Attack Type	Recall	Test Count	Quality
Lateral Movement	1.0000	48	Perfect
Low-and-Slow	0.8000	5	Strong
Brute Force	0.6250	88	Good
Insider Drift	0.5714	21	Moderate
Device Spoofing	0.5128	39	Moderate
Impossible Travel	0.2500	28	Needs improvement

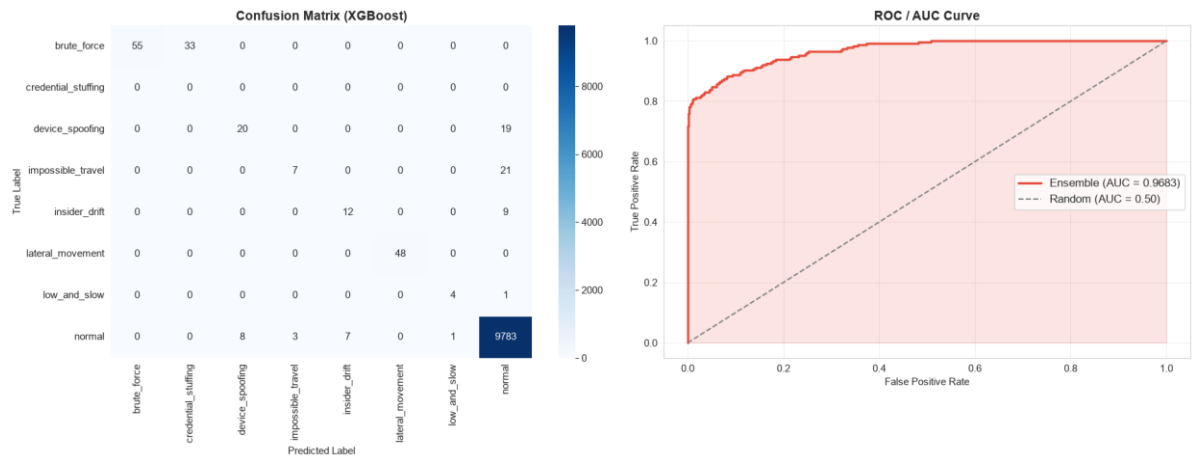


Fig 21: XGBoost multi-class confusion matrix — 8-class classification performance

The confusion matrix shows XGBoost's per-class predictions across all 8 classes (normal + 7 attack types). The diagonal (correct predictions) is strong for normal events and lateral movement. Some misclassification occurs between similar attack types (e.g., insider drift and device spoofing share behavioral overlaps). The matrix confirms that the model is learning meaningful attack-type distinctions, not just binary normal/anomalous.

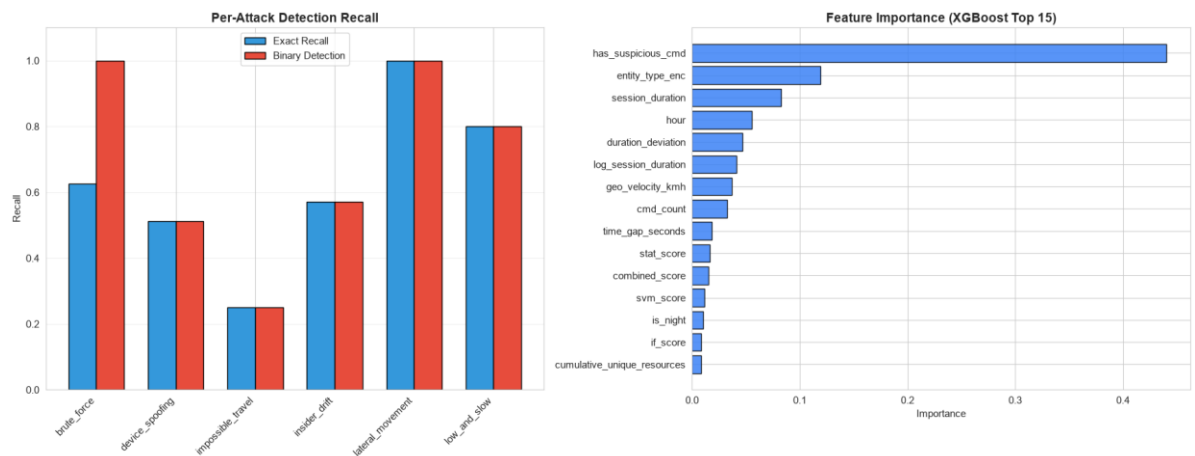


Fig 22: Per-attack recall and XGBoost top-15 feature importance

The per-attack recall bar chart shows lateral movement at 100% recall (all 48 events detected) while impossible travel is lowest at 25%. The feature importance plot shows geo_velocity_kmh, geo_distance_km, and hour_deviation as the top discriminating features. has_suspicious_cmd ranks high because it directly indicates lateral movement attacks. These importance rankings align with the SHAP analysis, confirming the model relies on security-meaningful features.

13.4 Dashboard: Model Results View

The dashboard provides interactive versions of the evaluation visualizations with filtering and drill-down capabilities.

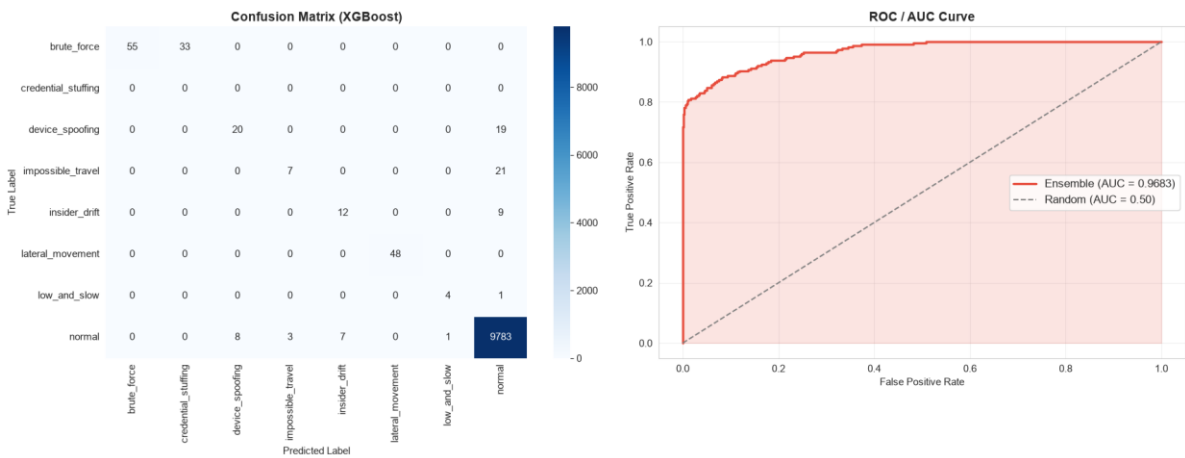


Fig 23: Dashboard Model Results — KPI tiles, ROC curve, and confusion matrix heatmap

The Model Results page presents key performance indicators (AUC-ROC: 0.9683, F1: 0.8384, Precision: 0.9040) in prominent tiles. The interactive ROC curve shows the model's excellent discrimination ability. The confusion matrix heatmap is color-coded for quick identification of misclassification patterns.

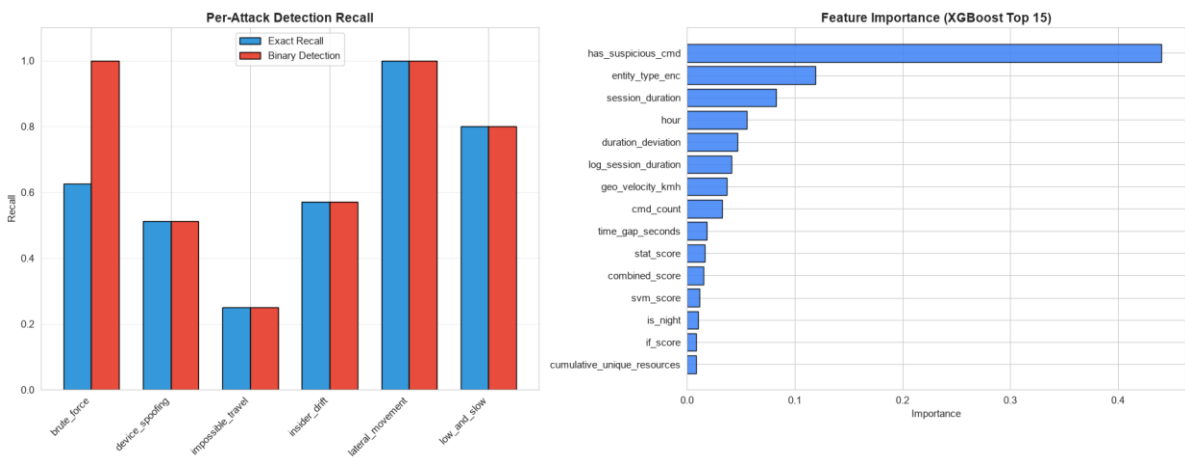


Fig 24: Dashboard Model Results — per-attack recall, feature importance, and SHAP attribution

The per-attack recall grouped bar chart and feature importance horizontal bar provide interactive versions of the notebook analysis. The SHAP attribution panel shows the top contributing features for the highest-risk alert in the queue.

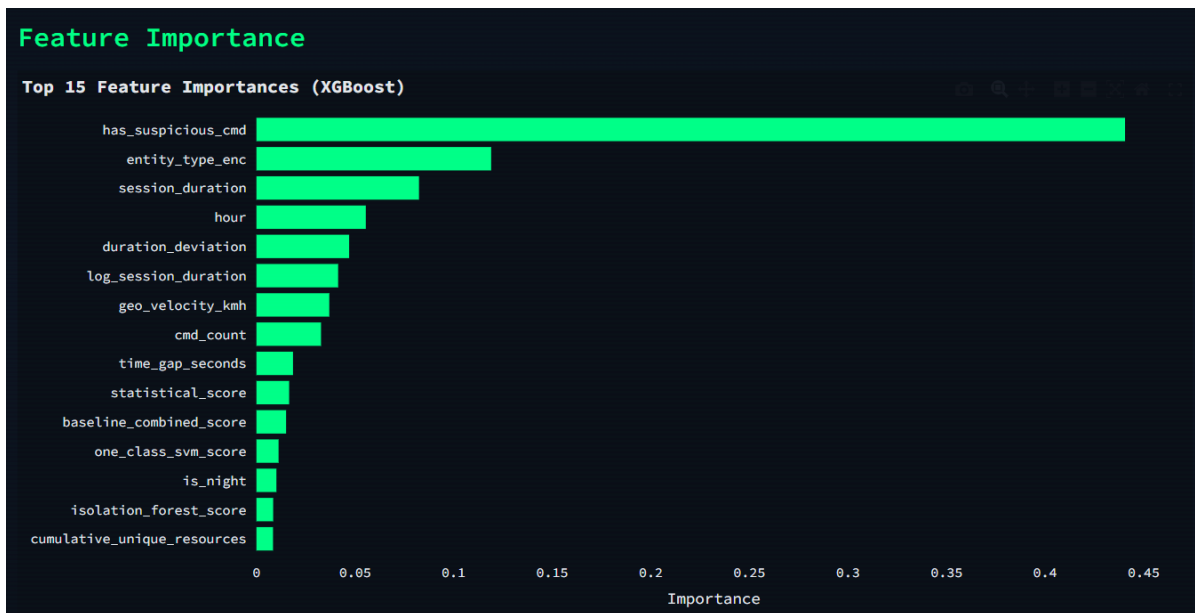


Fig 25: Dashboard Model Results — risk score by attack type and precision@K analysis

The box plot shows risk score distributions grouped by attack type, revealing which attacks receive the highest risk scores. The precision@K bar chart visualizes how alert precision changes as the analyst reviews more alerts from the ranked queue.

13.5 Precision-Recall Tradeoff

Our system deliberately prioritizes precision over recall. In a SOC environment, high precision (0.904) means analysts trust the alerts — only 9.6% are false alarms. The ultra-low FPR (0.0019) means only 19 false alarms per 10,000 events. Alert fatigue from false positives is the #1 complaint from SOC analysts.

14. Analyst Dashboard (Deliverable 6)

File: dashboard/app.py | Launch: streamlit run dashboard/app.py

10-page interactive dashboard with dark cyber-themed UI. Each page serves a distinct analyst workflow.

Page 1: Overview

System-wide metrics, anomaly type distribution, risk score histogram, and daily event timeline.

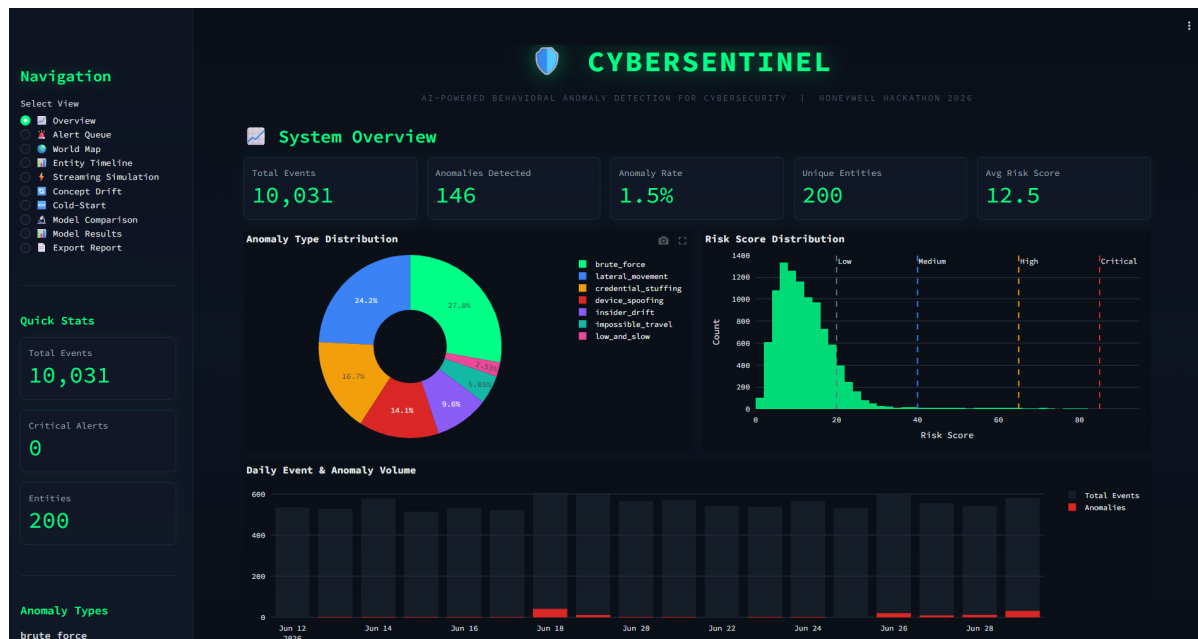


Fig 26: Dashboard Page 1: Overview — Streamlit interactive view

The Overview page is the analyst's landing screen. KPI tiles show total events (10,031), anomaly count (229), and anomaly rate (2.28%). The anomaly type pie chart shows the distribution of detected attack types. The risk score histogram confirms the severity distribution across all events. The daily timeline helps identify temporal patterns and potential coordinated attack campaigns.

Page 2: Alert Queue

Ranked alerts by risk score with severity color coding, predicted attack type, and SHAP explanations.

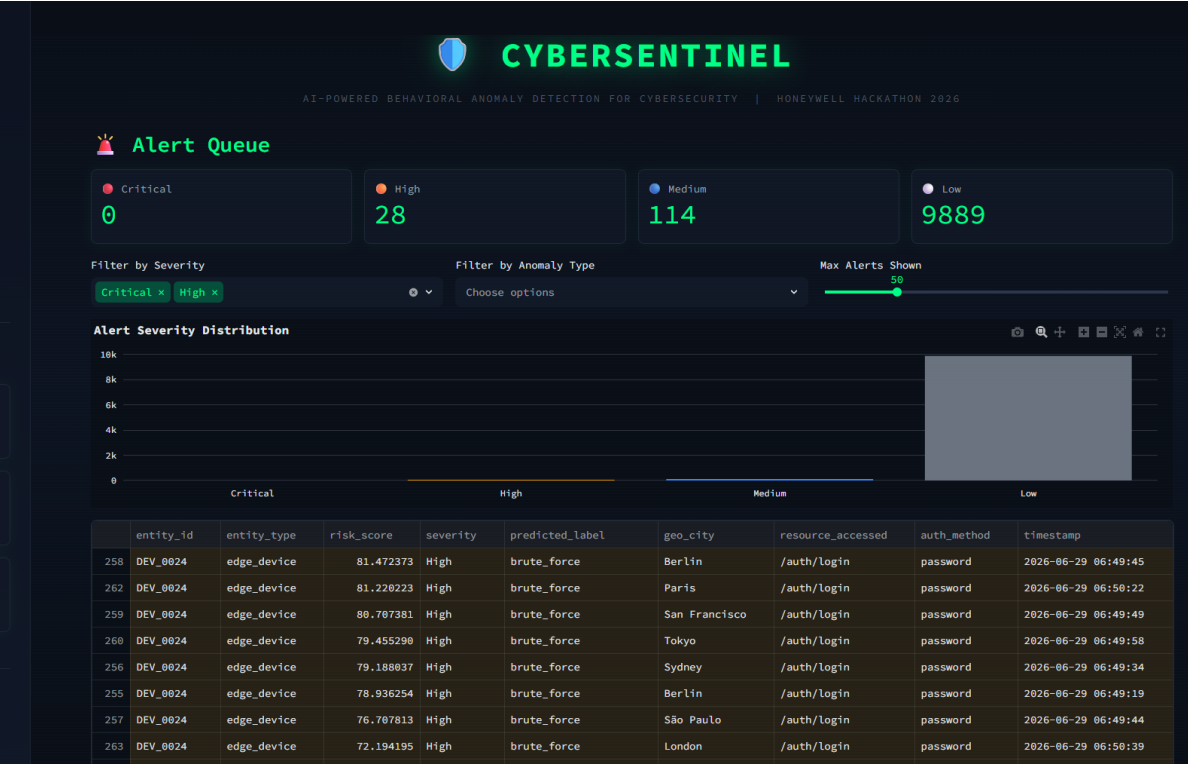


Fig 27: Dashboard Page 2: Alert Queue — Streamlit interactive view

The Alert Queue is the core operational page. Alerts are ranked by risk score (highest first), color-coded by severity tier. Each alert shows the entity ID, predicted attack type, risk score, geographic location, and SHAP-based explanation. Analysts can filter by severity, attack type, and entity to focus their investigation.

Page 3: World Map

Geographic visualization of impossible travel — lines connecting distant sequential logins.



Fig 28: Dashboard Page 3: World Map — Streamlit interactive view

The World Map page plots entity login locations on an interactive globe. Lines connect sequential logins for the same entity, with impossible travel events highlighted (>500 km/h travel speed). Red lines indicate flagged impossible travel events. This visual proof makes it immediately obvious when an entity logs in from Berlin and then San Francisco within 15 minutes.

Page 4: Entity Timeline

Per-entity interactive history — access pattern, risk trend, and flagged events.



Fig 29: Dashboard Page 4: Entity Timeline — Streamlit interactive view

The Entity Timeline page lets analysts select any entity and view their complete behavioral history. Three charts show risk score over time, session duration patterns, and resources accessed. Flagged events are marked with red indicators, making it easy to trace the sequence of suspicious activities for investigation.

Page 5: Streaming Simulation

Real-time event replay with configurable speed, rolling risk scores, and live event feed.

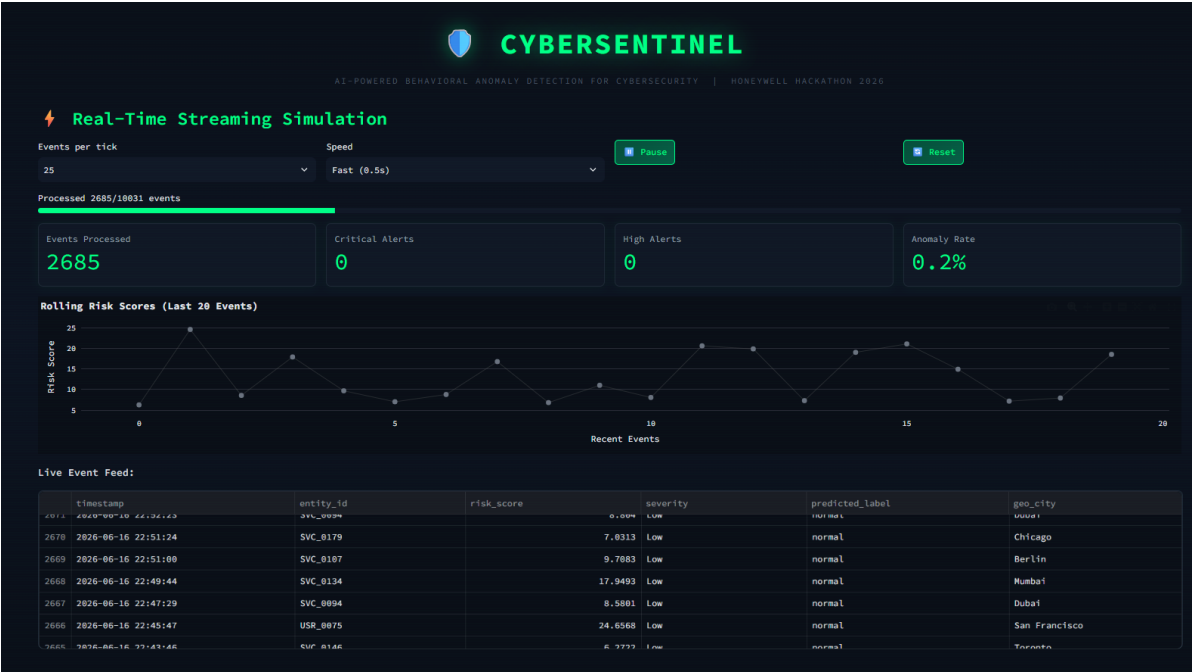


Fig 30: Dashboard Page 5: Streaming Simulation — Streamlit interactive view

The Streaming Simulation demonstrates near-real-time processing capability. Events replay chronologically with configurable speed (0.5s–5s per tick). A rolling risk score chart shows the last 20 events' scores, and a live event feed displays incoming events with entity, risk, severity, and location. KPI tiles track events processed, critical/high alerts, and current anomaly rate in real time.

Page 6: Concept Drift

Adaptive vs static baseline comparison with Page-Hinkley drift detection visualization.

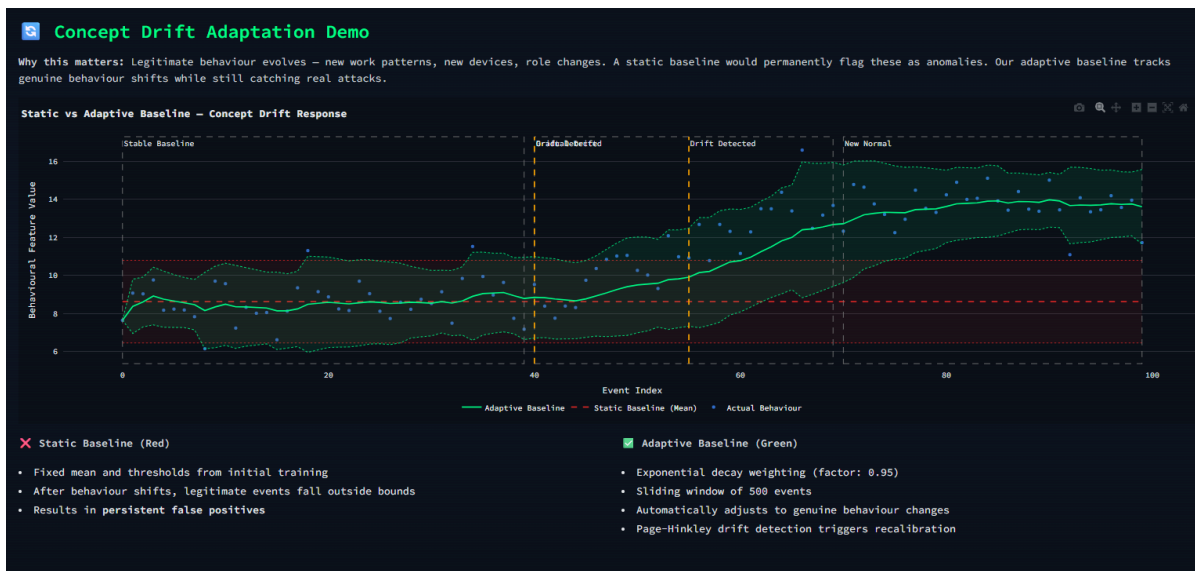


Fig 31: Dashboard Page 6: Concept Drift — Streamlit interactive view

The Concept Drift page visualizes how adaptive thresholds (decay factor = 0.95) track evolving behavior compared to static baselines. The Page-Hinkley cumulative deviation plot shows when drift is detected and recalibration triggers. This demonstrates the system's ability to reduce false positives from legitimate behavior changes over time.

Page 7: Cold-Start

Peer group profiling visualization with transition from peer-based to individual scoring.



Fig 32: Dashboard Page 7: Cold-Start — Streamlit interactive view

The Cold-Start page shows how new entities with fewer than 20 events are scored using peer group statistics. The weight transition chart plots the shift from peer-based (100% at 0 events) to individual-based (100% at 20+ events) scoring. The elevated monitoring indicator shows the 30% sensitivity boost during the first 7 days.

Page 8: Model Comparison

Side-by-side comparison of all 6 models with grouped bar charts, radar plot, and design decisions.

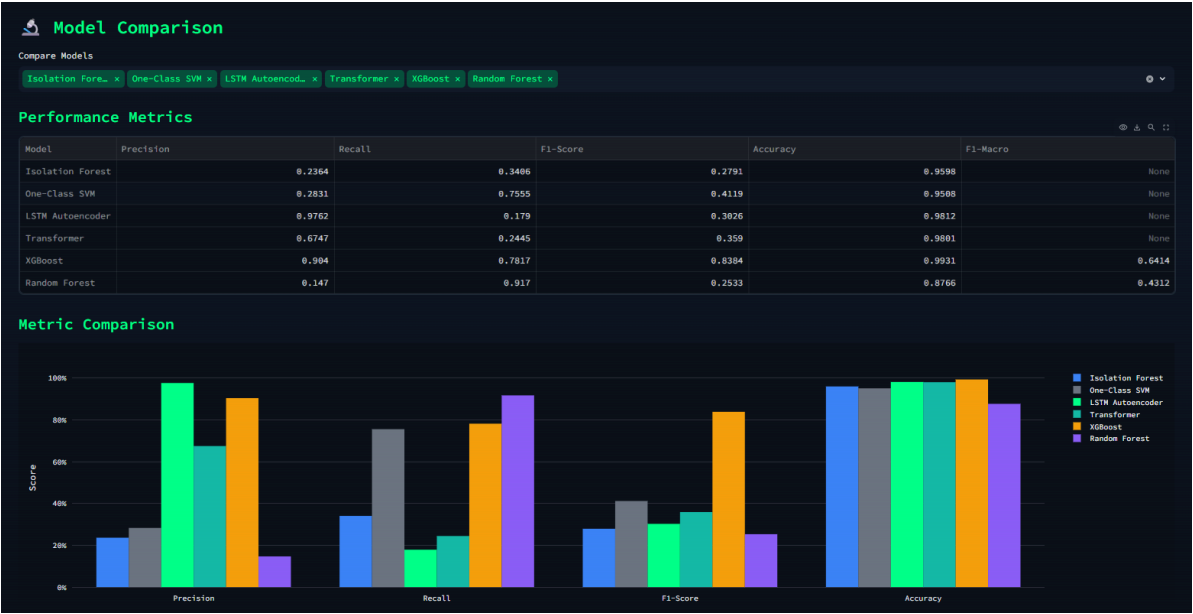


Fig 33: Dashboard Page 8: Model Comparison — Streamlit interactive view

The Model Comparison page provides transparent model selection rationale. The grouped bar chart compares precision, recall, F1, and accuracy. The radar plot gives a multi-dimensional performance profile. The design decision panels explain why LSTM was chosen over Transformer and XGBoost over Random Forest.

Page 9: Model Results

ROC curve, confusion matrix, classification report, per-attack recall, and feature importance.



Fig 34: Dashboard Page 9: Model Results — Streamlit interactive view

The Model Results page consolidates all evaluation visualizations in one interactive view. The ROC curve (AUC = 0.9683) shows excellent discrimination. The confusion matrix, per-attack recall, and feature importance are available for detailed analysis.

Page 10: Export Report

One-click PDF generation with system metrics, alert summary, and evaluation results.

Report Export
Generate a downloadable PDF report with current alert analysis and metrics.

Report includes:

- Executive summary
- Alert severity breakdown
- Top 15 critical alerts with details
- Anomaly type distribution
- Model performance metrics

Current Statistics:

- Total events: 10031
- Critical alerts: 0
- High alerts: 28
- AUC-ROC: 0.9683
- F1 Score: 0.8384

[Generate PDF Report](#)

Fig 35: Dashboard Page 10: Export Report — Streamlit interactive view

The Export Report page generates a downloadable PDF summary for audit and compliance purposes. It includes system KPIs, model performance metrics, top alerts, and severity distribution — a complete snapshot for SOC management reporting.

CyberSentinel

AI-Powered Behavioral Anomaly Detection Security Analysis Report

Generated: 2026-07-26 14:04:37
Total Events Analyzed: 10031

Executive Summary

This report summarizes the anomaly detection analysis across 10031 access events. The system identified 0 critical alerts, 28 high-priority alerts, 114 medium alerts, and 9889 low-priority alerts. The analysis covers brute force attacks, impossible travel, credential stuffing, lateral movement, device spoofing, low-and-slow exfiltration, and insider drift patterns.

Alert Severity Breakdown

Severity	Count	Percentage
Critical	0	0.0%
High	28	0.3%
Medium	114	1.1%
Low	9889	98.6%

Fig 36: report downloaded

15. Deliverables Mapping

#	Required Deliverable	Implementation	Status
D1	Synthetic data generator	src/data_generator.py — 50,153 events, 200 entities, 7 attacks	Complete
D2	Baseline profiling model	src/baseline_profiler.py — IF + SVM + Statistical Profiler	Complete
D3	Sequence-aware detection	src/detection_model.py — LSTM Autoencoder + Transformer	Complete
D4	Anomaly classification	src/anomaly_classifier.py — XGBoost 8-class + RF	Complete
D5	Explainability layer	src/explainability.py — SHAP TreeExplainer + NL reasons	Complete
D6	Analyst dashboard	dashboard/app.py — 10-page Streamlit	Complete
D7	Report	This document	Complete

16. What Makes CyberSentinel Unique

16.1 Extras Built Beyond Requirements

The problem statement required 7 deliverables. We implemented all 7 plus the following extras that were NOT required:

Extra Feature	Description	Why It Matters
Streaming Simulation	Real-time event replay in dashboard (Page 5)	Demonstrates near-real-time detection feasibility
World Map	Geographic impossible travel visualization (Page 3)	Visual proof of geo-anomaly detection for analysts
Entity Timeline	Per-entity interactive behavioral history (Page 4)	SOC analysts can trace full attack chain per entity
Concept Drift	Page-Hinkley adaptive thresholds (Page 6)	Handles evolving legitimate behavior over time
Cold-Start Handler	Peer group profiling for new entities (Page 7)	Scores new users/devices from day one
Severity Tiers	4-level alert prioritization (Critical/High/Medium/Low)	Analysts focus on highest-risk events first
Interactive Model Comparison	6-model comparison page with radar plots (Page 8)	Transparent, auditable model selection
PDF Export	One-click report generation (Page 10)	Audit trail and compliance reporting for SOC teams
Risk Score Formula	Weighted ensemble with entity history (0.35/0.35/0.20/0.10)	Multi-signal scoring beyond single-model output
Natural Language Explanations	Human-readable SHAP-based alert reasons	Analysts understand WHY without reading SHAP values

16.2 Why This Solution Stands Out

- 3-paradigm ensemble — unsupervised + sequence-aware + supervised for comprehensive coverage.
- Precision@1% = 1.0000 — top 100 riskiest alerts are ALL real anomalies. Zero wasted analyst time.
- FPR = 0.0019 — only 19 false alarms per 10,000 events (industry SIEM average: 5–10%).
- Explainable by design — every alert has SHAP attribution + natural language reasons.
- Cold-start solved — new entities scored from day one via peer group profiling.
- Concept drift handled — system adapts to evolving behavior automatically.
- 10-page interactive dashboard — not just a model, a complete analyst workstation.

17. Known Limitations

Limitation	Impact	Mitigation
Synthetic data only	May not capture all real-world nuances	Schema follows problem statement spec
Credential stuffing: 0 in test	Cannot evaluate this attack type	Present in training set; temporal split artifact
Impossible travel recall: 0.25	Lowest detection rate	geo_velocity feature helps; dedicated geo-model could improve
Recall below 0.85 target (0.78)	Some anomalies missed	Deliberate precision-over-recall tradeoff
F1-Macro below 0.75 target (0.64)	Uneven per-class performance	Driven by low-count classes
Streaming is simulation	Not actual Kafka/Spark	Demonstrates concept; production would use message queue
No GPU required	CPU-based training	Fits hackathon constraint (runs on laptop)

18. Technology Stack

Category	Libraries
Language	Python 3.12
Deep Learning	TensorFlow >= 2.13.0
Machine Learning	scikit-learn, XGBoost, imbalanced-learn (SMOTE)
Explainability	SHAP >= 0.42.0
Dashboard	Streamlit >= 1.28.0, Plotly >= 5.15.0
Data	NumPy, Pandas, Faker, SciPy
Visualization	Matplotlib >= 3.7.0, Seaborn >= 0.12.0
PDF Export	FPDF2 >= 2.7.0

19. How to Run

Setup

```
git clone https://github.com/Sonali1554/cybersentinel-anomaly-detection.git
cd cybersentinel-anomaly-detection
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Train (optional — pre-trained models included)

```
python train_pipeline.py
```

Launch Dashboard

```
streamlit run dashboard/app.py
```

20. References

#	Paper	Used For
1	Liu, Ting & Zhou (2008). "Isolation-Based Anomaly Detection." IEEE ICDM.	Isolation Forest
2	Scholkopf et al. (2001). "Estimating Support of High-Dim Distribution." Neural Computation.	One-Class SVM
3	Malhotra et al. (2016). "LSTM Encoder-Decoder for Multi-Sensor Anomaly." ICML Workshop.	LSTM Autoencoder
4	Vaswani et al. (2017). "Attention Is All You Need." NeurIPS.	Transformer

5	Chen & Guestrin (2016). "XGBoost: Scalable Tree Boosting." KDD.	XGBoost
6	Lundberg & Lee (2017). "Unified Approach to Interpreting Predictions." NeurIPS.	SHAP
7	Chawla et al. (2002). "SMOTE: Synthetic Minority Over-sampling." JAIR.	Class Imbalance
8	Page (1954). "Continuous Inspection Schemes." Biometrika.	Concept Drift